Principles of cybersecurity

Toghrul Karimov
Max Planck Institute for Software Systems, Saarbrücken, Germany

Last updated in 05.2026

Contents

1 Preliminaries

We write $x \,||\, y$ for the concatenation of two strings x and y.

1.1 Basic concepts from information security

The three core concepts of information security, called the "CIA Triad", are the following.

- *Confidentiality* means that information is accessible only to individuals to whom it is meant to be accessible. Example of non-confidentiality: you send a letter by post, but the envelope is opened in the middle, the letter is read, and sent to the recipient in an identical, fresh envelope.
- *Integrity* means that information is accurate, i.e. has not been tampered with/altered without the recipient's knowledge. Integrity is, in principle, independent of confidentiality: in the example above, it only means that the letter has cannot be modified in transit without the recipient noticing.
- *Availability* means that a system is available to legitimate users. Example of unavailability: you are trying to buy a concert ticket at the same time as thousands of other people, and the website is down.

We will add to these concepts throughout the course, starting with the following.

- *Authentication* means verifying the identity of the person performing an action, e.g. we can verify that the person sending the message is indeed the person with whom we intend to be communicating. Observe that authentication implies integrity. In the real world, we can authenticate a person using, for example, their passport: in doing so, we implicitly assume that passports are *infeasible to forge*, and, after assuming the document is genuine, *trust* the government organisation issuing it.
- *Non-repudiation* means that once an actor performs an action, they cannot deny having performed it. In the real world, once we sign a contract, we should not be able to deny that we did it.

1.2 Dolev-Yao model

Throughout this course, we model the attacker as being "as powerful as possible, within reason". In particular, we assume the attacker owns the network that transmits the messages, can store a large amount of messages from the past, has the physical capacity to send any message to anyone, and knows all the implementation details of our protocols (e.g., exactly which encryption protocol we are using) except the information that is explicitly secret (i.e., the secret keys). We additionally assume that the attacker has access to a large (but not infinite) amount of computational power that is commensurate with how much computation can realistically be performed at the time of writing. This model represents a well-funded, powerful adversary, and can be too conservative in various concrete situations. In those cases, we first construct an appropriate model of the adversary before analysing security of the relevant systems.

1.3 Basic Linux

Make sure you are familiar with the anatomy of Linux commands (command name, options, arguments), short and long forms of options, and how arguments are consumed by the options and the main command. Revise how absolute and relative paths work.

Every file is simply a sequence of bits. You can view a file in binary and ASCII (though not every byte corresponds to a printable ASCII character) using `xxd -b filename`. To view the contents in hexadecimal, use `xxd filename`.

To run a shell script, use `bash script-name`. The system variable called `$path` (in Windows it is called `$PATH`) contains a list of directories: when we run `command` $\cdots$ in the terminal, the shell searches through the directories in `$path` to find a file (i.e., the shell script describing what `command` should do) called `command`. To run a program from anywhere without needing to know its absolute path, either we add its parent directory to `$path`, or move its code to a directory that is already present in `$path`. Try `echo $path`.

Install, update, and uninstall programs using `apt` (advanced package tool), which is the main package manager for the Debian family of Linux distributions. (In MacOS, I use `brew`.) For this course, to install Python and Flask you can do a global install using `sudo apt update; sudo apt install python3` and `sudo apt install python3-flask`, without worrying about `pip` or virtual environments.

Feel free to use `whereis`, `which`, `find` to locate things in your file system: when searching for a file by its name, you can use something like `find / -name filname 2>/dev/null` to not print all error messages that arise e.g. when you do not have permissions to open a directory.

A good book is "The Linux Command Line" by William Shotts, available online for free.

1.4 Homework

1) Study the commands `grep` and `awk`. Apply them to your own examples.

2 Hash functions

2.1 Definitions and properties

A *hash function* is a function f that takes a binary string of arbitrary length, and outputs a binary string of fixed length ℓ. For example, `SHA2` (SHA stands for "secure hashing algorithm") always outputs a binary string of length 256. A desirable property of a hash function is that, for every bit length n, the set of all images of f on strings of length n should be uniformly distributed. That is, for every n and binary string y of length ℓ, the number of binary strings x of length n such that $f(x) = y$ should be close to $2^n/2^\ell$. (Recall that 2^n is the number of different strings of length n, and 2^ℓ is the number of possible outputs of f.) Outside of cryptography, hash functions are used in programming, most notably in *hash tables*.

An example of a hash function is $f(x) = x \bmod 2^\ell$, where we convert back and forth between numbers and their binary representations to perform the remainder operation. The outputs of this f are, in fact, exactly uniformly distributed. Another example of a hash function is f that squares the input and returns the middle ℓ bits from the resulting binary representation.

A *collision* for f is two distinct inputs x_1, x_2 such that $f(x_1) = f(x_2)$. A *cryptographic hash function* is hash function with extra properties related to their safety. These are as follows.

1. Preimage resistance. The problem of finding, given a string y of length ℓ, a string x such that $f(x) = y$ is *computationally infeasible*.
2. Second preimage resistance. The problem of finding, given x_1, y such that $f(x_1) = y$, a string x_2 such that $f(x_2) = y$ is computationally infeasible.
3. It should be *computationally infeasible* to generate new collisions for f.

When the properties (1-3) are satisfied, then the function f should be essentially "random". In particular, making a small change to x should completely change $f(x)$. We mention that all hash functions (whose input size, by our definition, is unbounded) have infinitely many collisions, and a collision can always be found in principle: just try $2^\ell + 1$ different values of x. The point is that, in a secure hash function this should essentially be the only way to search for collisions, and it should,

with very high probability, require a very large amount of time (e.g. decades) before a collision is found.

For example, `SHA2` does satisfy the properties (1-3) above: in fact, not a single collision is publicly known for it. The earlier version `SHA1` (which produces 160-bit outputs) does not satisfy (3); in fact, it has been cracked in a stronger sense. A *chosen prefix attack* has been successfully implemented, meaning that it is computationally feasible, given x_1, x_2, find two strings z_1, z_2 such that $f(x_1 \,||\, z_1) = f(x_2 \,||\, z_2)$, where $x \,||\, z$ denotes the concatenation of x and z. Today, `SHA1` is not considered secure and not used any more (read `https://en.wikipedia.org/wiki/SHA-1#Attacks` and `https://security.googleblog.com/2017/02/announcing-first-sha1-collision.html`). The hash function $f(x) = x \bmod 2^\ell$ we gave earlier does not satisfy any of the properties (1-3) above, and should not be used in any cryptographic application.

A common way to build cryptographic hash functions is through the *Merkle-Damgård construction*. We illustrate it for `SHA2`. Let g be a *one-way* function (meaning that it is computationally infeasible to invert it) that takes two inputs a, b of bit lengths 256 and 512, respectively. Further let b_0 be a fixed string of length 256, called the *initialisation vector* of the hash function. To hash an input x, we first pad it (i.e., extend it in some pre-defined way) to a word x' whose length is a multiple of 512. Write $x' = a_0 \,||\, a_1 \,||\, ... \,||\, a_m$, where each a_i has length 512. To compute the hash, we perform

$$\begin{aligned} b_1 &= g(a_0, b_0) \\ b_2 &= g(a_1, b_1) \\ &\vdots \\ b_{m+1} &= g(a_m, b_m) \end{aligned}$$

and output $y = b_{m+1}$.

Successful attacks on hash functions typically exploit its (block) structure; this is an example of *cryptanalysis*. A *brute-force attack*, on the other hand, does not use any information about f: rather, it tries to find a collision by generating $(x, f(x))$ pairs until a collision is found. A brute-force attack on a hash function is also called a *birthday attack*. The name comes from the *birthday problem*, which is as follows. Suppose we choose n people randomly. (Think of a person as a binary string and their birthday as the corresponding hash.) What is the probability that two people share the same birthday? It turns out that for $n = 50$, this probability is already $\approx 97\%$, despite 50 being much smaller than the number 366 of all possible birthdays; see `https://en.wikipedia.org/wiki/Birthday_problem`. In the context of hash functions, by the same mathematical analysis, it is known that to find a collision for an ℓ-bit hash function, we need to generate $2^{\ell/2}$ pairs $(x, f(x))$ *on average*.

2.2 Applications of hash functions

Hash functions have many applications in cryptography. We next discuss one of them: storing passwords. When you register an account on any website, nobody except you should now your passwords. Similarly, passwords should never ever be stored, due to the possibility of the database of passwords getting compromised. How can we then perform password-based authentication? The answer is: the authenticator stores not the password but its hash. When the user logs in, the hash of the entered string is computed and compared against the stored hash. Because cryptographic hash functions are preimage and collision-resistant, even if the database of stored password hashes gets stolen, the attacker will not be able to gain access to user accounts by breaking the hash function.

However, the protocol for storing passwords described above is still not secure. The reason is that the passwords used by real users are typically not random strings–rather, they often come from a relatively small list of "natural" strings. Hence an attacker crack at least some passwords by comparing the hashes in the password database against the hashes of all commonly used passwords (stored in an efficient, special data structure called *rainbow table*). Hence one should store *salted hashes* instead of just hashes. A salt is additional, *random* information that is added it to the password, which could, for example, be the date and time of password creation. That is, we store s and $f(p \,||\, s)$ on the database, where p is the password and s is the salt. This precludes the use of rainbow tables, even if the attacker knows all the salt values.

Another application of hash functions consists of *checksums*. When software is published, it usually comes with a way of checking the integrity of the files to be downloaded. See, for example, "File Checksums" in `https://www.virtualbox.org/wiki/Downloads`. When you download VirtualBox, it is your responsibility to compare the hash of the file you downloaded against the hash value provided

by the publisher of the software: it is possible that an attacker has swapped (for example, somewhere in the network) the code you wanted to download with harmful code under the same file name.

2.3 Programming examples and homework

In a Linux-based system, the command-line program `sha256sum` computes the `SHA2` hashes (in hexadecimal); see `https://man7.org/linux/man-pages/man1/sha256sum.1.html` for the manual. (In `PowerShell`, you would use `Get-FileHash`.) To compute the hash of a file, use `sha256sum filename`. You can also compute hashes of multiple files: `sha256sum filename1 filename2 filename3` To compute the hash of a string, use `printf string | sha256sum` or `echo -n string | sha256sum`; this is called *piping*, which chains command-line instructions. You can get the hash of a string `s` by doing `printf s | sha256sum | awk 'print $1'`.

You can use the `--check` (or `-c` for short) option of `sha256sum` to verify checksums. To do this, you first create a file `hashes.txt` (can be called anything) of rows of the form `filename checksum` (with exactly two spaces between the two) where `checksum` is the correct hash of the corresponding file. Then run `sha256sum --check hashes.txt`. This goes over every line in `hashes.txt`, hashes the file called `filename`, and compares the result against `checksum`. Try `printf hello > message.txt; sha256sum message.txt > hashes.txt; sha256sum --check hashes.txt`.

In a Linux system, you can find the hashes of the user passwords (together with the corresponding salts) in the directory `etc/shadow`. (You can find more password information under `etc/passwd`.) You need root permission to access the file: `sudo cat etc/shadow`. In Linux Mint, rather than SHA256, yescrypt is used by default for password hashing.

We move on to `hashcat`, which is a command-line program that efficiently performs (using the GPU of a computer) a large number of hashing operations at the same time. It can be used to "crack" a hash. Fundamentally, it takes a hash y (stored in a file) and a list of candidate words x, and checks for all x whether $f(x) = y$. You can install `hashcat` using `sudo apt install hashcat`. See "General guides" at `https://hashcat.net/wiki/` for documentation. For example, `hashcat -m 1400 -a 0 hashes.txt wordlist.txt` performs a dictionary attack (because we say `-a 0`) using `SHA2` (because we say `-m 1400`) on hashes stored in `hashes.txt` using a list of candidate inputs `wordlist.txt`.

Homework 1) Use `sha256sum` with your own examples. 2) Look up `rockyou.txt`. 3) Investigate the full capabilities of `hashcat`.

3 Encryption

The goal of encryption is to send a message without revealing any of its contents to anyone other than the intended recipient. *Plaintext* refers to the message we want to send; we denote it by m. We use an *encryption key* (denoted k_e), to compute the *ciphertext* $c = f(m, k_e)$, where f is the *encryption algorithm*. Once the recipient receives the ciphertext, she *decrypts* it using the *decryption key* k_d: writing g for the *decryption algorithm*, we should have that $g(f(m, k_e), k_d) = m$ (as well as $f(g(m, k_d), k_e) = m$) for every possible string m and key pair k_e, k_d. We mention that *end-to-end encryption* means that the message is encrypted by the sender's device and decrypted only by the receiver;s device, at the very end of its transmission.

3.1 Symmetric-key cryptography

In this case, encryption and decryption are both performed using the same key $k = k_e = k_d$, which must be a secret between the sender and the intended recipient. There are two kinds of symmetric-key cryptographic algorithms: *block* and *stream* ciphers. In a block cipher, the plaintext is divided into blocks, and each block is encrypted (and later decrypted) independently, all using the same secret key. On the other hand, in a stream cipher, we first generate a *keystream* s from the secret key that is as long as the plaintext, and then compute $m \oplus s$, i.e. the `XOR` of the plaintext and the keystream. We can always turn a block cipher into a stream cipher: the latter is a strictly large family of algorithms.

3.1.1 Block ciphers

The simplest class of symmetric-key encryption algorithms are the *substitution ciphers*. In this case, we replace each unit of plaintext (for example, every letter or every byte) with another one according

to a prescribed rule. The secret key is then the (finite) list of replacement rules. *Caesar cipher* is an example of a substitution cipher. In a substitution cipher, both the encryption and decryption algorithms are the same. To see this, suppose we are working with strings of a, b, c, and encrypt plaintexts using the substitution $a \to b, b \to c, c \to a$. Then decryption with this key is the same as encryption with the key $a \to c, b \to a, c \to b$. Substitution ciphers can have a large number of possible keys (e.g., we can replace letters of the English alphabet according to $26! \approx 2^{88}$ different possible rules), but can be easily attacked using a *frequency analysis*. For example, we know that by far the most common letter in English is `e`. Therefore, if we have access to a large amount of ciphertext, even if we don't know the key k we can still determine the letter to which `e` is mapped by counting frequencies of letters in the ciphertexts. Once we identify the letter to which `e` is mapped, we can move on to another letter. See `https://en.wikipedia.org/wiki/Letter_frequency` for relative frequencies of each letter in English texts.

A more modern encryption algorithm is AES (Advanced Encryption Standard), standardised by NIST in 2001. It always divides the plaintext into blocks of 128 bits (padding the input to a multiple of 128 at the beginning if necessary), but can have 128, 196, or 256 bit keys. In this course, we work with AES256. Pure AES (referred to as AES in the *block mode*) operates on each 128-bit block b as follows, computing a 128-bit output. First, write $b = c_1 \,||\, \cdots \,||\, c_{16}$, where each c_i is a single byte. Then it arranges the 16 bytes into a 4×4 matrix. Thereafter, it performs 14 rounds of the following: perform a complicated but invertible operation on the matrix, derive a fresh round key k' from the secret key k, and take the `XOR` of the matrix with k'. No round key is re-used. Note that if we know k, then we know every round key k', and we can perform decryption because `XOR` with k' is an invertible operation: $b = (b \oplus k') \oplus k'$.

All pure block ciphers can be attacked using frequency analysis. Hence in practice, we always have some modification. AES itself has 4 other such modes: AES-GCM, AES-CBC, AES-CTR, AES-CCM.

3.1.2 Stream ciphers

We first give `AES-CTR` with 256 bit keys as example. We write $\text{AES}(k, b)$ for the output of AES in the block mode (see the section above) on a 128-bit block b with key k. Again, we split the (padded) plaintext m into 128-bit blocks $b_1, \ldots, b_l$. Next, we generate a 96-bit random string r, called the *nonce* (number used only once). Thereafter, we generate the keystream

$$s = \text{AES}(k, (r \,||\, 1)) \,||\, \cdots \,||\, \text{AES}(k, (r \,||\, l))$$

which has exactly the same length m. Note that so far we have not used the plaintext at all: we have just encrypted the nonce together with the block numbers, called the *counters*. To produce the ciphertext, we then take the `XOR` of the keystream with the plaintext, i.e. $c = m \oplus s$. Importantly, we transmit the ciphertext c together with the nonce r: the latter is necessary to decrypt the message. (Compare the nonce to the salt used in hashing.) The receiver, upon receiving (c, r), first computes the keystream s from the shared key k and the nonce c, and finally computes

$$m = c \oplus s = (m \oplus s) \oplus s.$$

If the same nonce is re-used with the same key, then we are back to being vulnerable to frequency analysis. However, observe that the space of possible nonces is large with size 2^{96}.

Other modes of AES can provide *authentication* in addition to *confidentiality*. The AES-GCM mode, for example, transmits three pieces of data: the ciphertext, the nonce, and a 128-bit *tag*. The tag is computed from the ciphertext, the nonce, and the secret key using a hashing algorithm called GHASH. We can then verify the identity of the sender by comparing the tag against $\text{GHASH}(c, r, k)$. This is called Authenticated Encryption with Associated Data (AEAD).

3.2 How do we establish a shared key in the first place?

Historically, shared secrets between two or more parties were established through separate safe channels: for example, the key was printed on a piece of paper, put into a sealed envelope, and then personally delivered to the other party. (See `https://en.wikipedia.org/wiki/Cryptanalysis_of_the_Enigma#Key_setting`.) However, using some interesting mathematics it is possible for two people to establish a secret key while communicating only over a public channel. We will discuss the *Diffie-Hellman key exchange protocol*, first published in 1976.

The protocol works as follows. Suppose Alice and Bob want to establish a secret k of length l bits. They first publicly agree on a prime number p of bit length l, and a base $1 < g < p$. Then Alice generates a secret number $0 \leq a < p$, computes $A \coloneqq g^a \bmod p$, and transmits A to Bob. Similarly, Bob generates a secret number $0 \leq b < p$, computes $B \coloneqq g^b \bmod p$, and transmits B to Alice. Thereafter, Alice computes $k = B^a \bmod p$ and Bob computes $k = A^b \bmod p$; in the end, their shared secret is $g^{ab} \bmod p$. The values computed by Alice and Bob are the same thanks to Fermat's Little Theorem: for every integer x and a prime p, $x^p \bmod p = 1$. This protocol is considered secure because the following problem, called the *discrete logarithm problem*, is computationally intractable: given g, A, p, find a (which generally is not unique) such that $g^a \bmod p = A$. Essentially, the attacker has to try values of a until one is found. (Note: some choices of g, p are better than others, with respect to the computational difficulty of the corresponding discrete logarithm problem: `openssl`, for example, when performing classical Diffie-Hellman chooses $g = 2$ and p such that $(p-1)/2$ is also a prime.)

3.3 Asymmetric-key cryptography

In this case, we have two different but mathematically related keys: k_e is the *public key*, used for encryption, and $k_d \neq k_e$ is the *private key*, used for decryption. The main advantages of asymmetric-key cryptography are that (i) it solves the key distribution problem (no need to establish secrets at the beginning of a communication, as the public keys can be shared openly), and (ii) it provides authentication/digital signature. The main disadvantage is that it is slow compared to symmetric-key algorithms. Hence asymmetric-key protocols are often used to establish a symmetric *session key*.

We discuss RSA, the most classical asymmetric cryptographic algorithm, named after its inventors Rivest, Shamir, and Adleman. Recall that Diffie-Hellman algorithm was based on computational difficulty of the discrete logarithm problem. RSA, on the other hand, is based on the computational difficulty of factoring large numbers.

In the context of RSA, everybody has a public key k_e and a private key k_d. We generate such a pair as follows. First generate two large prime numbers p, q (each around 1000-2000 bits long), and set $n = pq$. Then compute $\varphi = (p-1)(q-1)$. Next, choose $1 < e < \varphi$ such that $\gcd(e, \varphi) = 1$. In particular, we can choose e to be any prime in $(1, \varphi)$. In most implementations of RSA, e is chosen to be 65537. Finally, use the extended Euclidean algorithm (which is computationally very efficient) to find d such that $e \cdot d \bmod \varphi = 1$. Then the public key is (n, e) and the private key is (n, d). Note that if we could factor n as $n = pq$, then we could immediately compute the private key from the public key.

We make (n, e) openly available, and when someone wants to send a message m to us, they encrypt it as follows. We suppose that, when interpreted as a number, $m < n$; otherwise we have to send multiple messages. The ciphertext (which is sent to us) is simply the (binary representation of) $c \coloneqq m^e \bmod n$. (We mention that, due to our choice of e, every different $0 < m_1, m_2 < n$ are mapped to different ciphertexts, i.e. $m_1^e \bmod n = m_2^e \bmod n$ only when $m_1 = m_2$.) To decrypt c, we simply compute $m = c^d \bmod n$. This works because of laws of modular arithmetic.

Still, RSA is never used in practice in its raw form described above. The reason is that it lacks randomness: it is possible, for example, to attack signatures of small/common messages because the same messages are always signed/encrypted in exactly the same way. Modern implementations of RSA never encrypt/sign only the message m, but rather a transformation of m that involves a random string (which is then transmitted along the ciphertext/signature).

Is it better to every time establish a session key using Diffie-Hellman, or exchange messages using RSA? The former provides *forward secrecy*, whereas the latter does not. Forward secrecy means that if a key (e.g. the private key in RSA) gets compromised, then the attacker (who can store previous messages) is not able to decrypt past messages. In case of Diffie-Hellman, each session has its own key; hence a loss of one key does not compromise messages from other sessions.

However, unauthenticated Diffie-Hellman is weak to the *man-in-the-middle attack*. Here, Alice and Bob think that they are establishing a session key, but instead end up each establishing a session with Eve, who can then relay and modify the messages between Alice and Bob as she pleases. The modern way is to establish the *ephemeral* Diffie-Hellman key over an (RSA-)authenticated communication channel. Question: Suppose one party generates a session key and sends it to the other one using RSA encryption. Is this a good protocol? Why bother with doing both RSA and Diffie-Hellman?

RSA is not actually considered the strongest nor the most modern asymmetric-key algorithm: *elliptic curve cryptography* is considered stronger, and needs significantly smaller key sizes. However,

both families of algorithms are insecure against *quantum computers*. Completely new and even stronger algorithms have been proposed that will remain secure even if we manage to develop practical quantum computers; the area that studies this is called *post-quantum cryptography*.

3.4 Digital signatures

The reverse of the process described above can be used to implement *digital signatures*. Suppose I need to sign a message m. First, I compute the hash of m, written h. Then I compute the signature $s = h^d \bmod n$, and transmit it along m and h. Upon receiving (m, h, s), the recipient (i) calculates the hash h' of m, and compares it with h, and (ii) computes $s^e \bmod n$ and compares the result with h. If everything matches, then it means that the message was undeniably sent by me. This protocol provides *authentication* and *non-repudiation*. (We mention that symmetric-key protocols do not provide non-repudiation, because the key is known to two people: thus either person can plausibly claim that the action was performed by the other.) In summary, RSA works because by the properties of p, q, e, d, for any m we have that

$$(m^e \bmod n)^d \bmod n = m = (m^d \bmod n)^e \bmod n,$$

and the problem of determining the value of d only knowing n and e is computationally infeasible.

How are public keys bound to real-world identities? One approach is that, when I claim to someone to be the entity X, I present a certificate digitally signed by a trusted *certificate authority* (CA). The CAs verify the identity (using various means: for example, to verify that you own a domain name they can ask you to create a certain page under your domain name) when producing the certificate, but also have the power to revoke a certificate when, for example, a private key gets compromised.

A second approach (called "web of trust") is the fully distributed one employed by PGP (which stands for *pretty good privacy*). In this setting, there is no central verifier: all entities sign the keys that they trust. When deciding whether to trust a key, we either look at the circumstantial evidence (I trust the website on which the public key is published), or check whether anyone we already trust has signed the public key in question. PGP is widely used in signing (and encrypting) emails.

3.5 Message authentication codes (MAC)

We saw that AES-GCM computes, in addition to the ciphertext, a tag. This is an example of a MAC. Encryption provides confidentiality, but it does not provide integrity. MACs, on the other hand, provide integrity and authentication (but no confidentiality on their own without accompanying encryption). MACs can be constructed from a hash function (HMAC; hash the key and the message together), or a cipher (CMAC; like a block hash function, but instead of a one-way function, use a block cipher internally).

When using MAC together with encryption, the correct way is to first encrypt the message m to get the ciphertext c, and then compute the MAC for c, not the other way around. This way, the receiver first checks the MAC for the whole message. If the check fails, the she does not decrypt the message at all: this is a defence against chosen-ciphertext attacks.

We mention that confidentiality + integrity is still not everything: the attacker can store a valid combination of a ciphertext and a MAC, and send it (for the first time or again) at a later time. This is called a *replay attack*. To prevent this, messages can include, for example, the date and time or the session number. The term for being safe from a replay attack is *freshness*: it means that the message is fresh and not from the past.

3.6 Programming examples and homework

We can perform AES-CTR encryption and decryption using `openssl`. (It does not, however, support AES-GCM or any other AEAD modes.) To encrypt, we use `openssl enc -aes-256-ctr`. It expects (i) the plaintext, either from `stdin` or via the optional argument `-in <filename>`, (ii) a key in the form `-K <256 bits as hex characters>`, and (iii) a 128-bit *initialisation vector* (which has the same function as nonce+block counter) in the form `-iv <128 bits as hex characters>`. It outputs a binary string, which you can redirect to a file using `>`. Alternatively, you can specify an output file using the `-out` option. Decryption works in exactly the same way, but you additionally use the `-d` option. We can use shell variables to store the key and the nonce: for example, we

can do `KEY=$(openssl rand -hex 32)` to generate and store 32 random bytes (represented as hex characters), and then access with `"$KEY"`.

One can generate a public-private RSA key pair using `ssh-keygen -t rsa`. Usually it will be stored in the (hidden) folder `~/.ssh` (where the dot means the file is hidden). You can view the contents of this folder using `sudo ls -a ~`; the option `-a` ensures that the hidden files are also listed. If you do `sudo ls ~/.ssh`, you should see the files `id_rsa` and `id_rsa.pub`. These are the private and public keys, respectively, which can be viewed using `cat`. RSA encryption and decryption (as well as RSA digital signatures) can be performed using `openssl-rsautil` (see `https://docs.openssl.org/master/man1/openssl-rsautl/`) or `openssl-pkeyutil` (more modern, see `https://docs.openssl.org/3.0/man1/openssl-pkeyutl/`). You can use `ssh-keygen` to convert your keys into the PEM format.

Homework. 1) In AES-CTR, what if we do not generate a nonce? That is, what would happen if we use just AES(1) || AES(1) || $\cdots$ as the key stream? 2) Investigate the modes of AES that we did not discuss. 3) Investigate how Diffie-Hellman key exchange algorithm is implemented in practice, e.g. how large prime numbers are found and how $g^a \bmod p$ is efficiently computed. 4) Look up the following attack modes: ciphertext-only attack (COA), known-plaintext attack (KPA), chosen-plaintext attack (CPA), chosen-ciphertext attack (CCA). (Note: when discussing security of protocols, it is common to assume at least a CPA and usually a CCA attacker.)

4 Operating systems

Different users of the same system share its computing resources, which include memory, the file system, CPUs, networks, peripherals etc. The term *authorisation* refers to what each user is allowed to do. In this section, we discuss how authorisation works in a Linux system.

4.1 Discretionary access control

The traditional access control model of a Linux system is called *discretionary access control* (DAC). Here we have users and groups (which are collections of users with common permissions). Each running process has a user ID (UID), which identifies the user to which the process belongs, as well as one or more group IDs (GIDs), which record all groups to which the owner of the process belongs. Similarly, each file has an associated UID (the identity of the owner), a GID (the group of the file), as well as a list of permission for the owner, the group, and everyone else. The things a process is authorised to do to a file is determined by checking the UIDs and the GIDs associated with the process and the file in question.

The permissions associated with a file are described by a 10-letter string $x_1 \cdots x_{10}$, where each letter except the first one is either `r` (read), or `w` (write, i.e. modify), or `x` (execute, i.e. run), or `-` (no permissions). Here

1. x_1 (`-` for a regular file, `d` for a directory file, i.e. a folder, and one of `b`, `c`, `p`, `l`, `s` for special files) describes the type of the file;

2. $x_2x_3x_4$ describes the permissions of the owner, e.g. $x_2x_3x_4$`=rwx` means the owner can read, write, and execute;

3. $x_5x_6x_7$ describes the permissions of the users belonging to the group of the file; $x_8x_9x_{10}$ describes the permissions of everyone else.

4.2 The root and other users

The directory `/` is called the *root* of a (Linux-type) file system. The whole file system is organised under `/`. Additionally, each user has the *home directory* `/home/username`, which can also be accessed using the shorthand `~`. (In MacOS, despite it being Linux-based, the home directories of all users are stored in `/Users`.) There is a special user called the *root user*, which typically has unlimited power. Usually, no actual (human) user is actually logged in as the root user: rather, there is a command called `sudo` (which means *superuser do*) through which a select set of users can temporarily give themselves root user privileges. (An account with full `sudo` privileges can also configure what other users can do using `sudo`.) For example, the first user of a Linux system (typically the system

administrator) can do everything that the root user can do using `sudo`. If you are truly logged in as the root user (which usually does not happen or is not allowed due to security reasons), then you don't need `sudo`.

Any user with root privileges can create a new user, delete an existing user, change their password/groups, view the contents of all files, control the network access of other users etc.. (Having root privilege means having UID zero, which is not the same as having `sudo` access: what each user can do using the `sudo` command can be configured by the root user.) It is possible to restrict what the root users can do using *mandatory access control* (MAC).

A user with root privileges can also *audit* other users: this includes reviewing all of their activity, including logins, commands executed etc. See `auditid`.

4.3 Programming examples and homework

The conventional way to give a file permission to run is `chmod +x filename`. You can view the list of all running processes, as well as the associated UIDs/GIDs using the command `ps`. To do the same for a file in the current directory, use `ls -l`, and for a file located at `path`, use `ls -l path`. The command to change permissions of a file is `chmod`. You can create a new user using `sudo adduser username`. The newly created user by default does not have any `sudo` access. To remove an existing user, use `sudo deluser`. You can open a terminal, logged in as another user, using `su` (switch user). The password of the user running the current shell can be changed using the `passwd` utility.

Homework 1) Create a new user and add a file to their home folder using the terminal. 2) Then log in to the new user and view the file. Go back to the root user and delete the file, and log in to the new user to confirm that it is not there any more.

5 Basics of the internet

The internet is a distributed system for exchanging information that is not owned by a central body. (Compare internet with the world wide web.) Understanding its architecture and protocols is important for ensuring computer security.

5.1 Client-server architecture

A *host* is just a computer connected to a network, usually the internet. In one host, we typically run many different *client* and *server* processes. A server is a process that waits for a request from a client, and then sends back a response. (In particular, it does not contact potential clients first.) Servers are often reachable from anywhere on the internet, but they can also be accessible only from inside a smaller network. A client, on the other hand, initiates the connection by sending a request to a server. Not being reachable from anywhere is generally not a problem for a client: the important thing is that the server should be able to somehow get back to the client once the communication has begun. Servers can serve different things: for example, they can serve the files necessary to display a website (a web server), or information about how to reach the server associated with a domain name supplied by the client (DNS server). The client-server architecture is ubiquitous in all sorts of computer networks, but is not the only possibility: consider, for example, how radio devices work.

In the client-server framework, a *proxy* is someone that communicates on behalf of the client (the client tells proxy what it wants, the proxy does it, and passes on the result to the client), and *reverse proxy* is someone that communicates on behalf of the server (the client thinks it is talking to the server, but it is talking to a reverse proxy who relays the requests and responses). Thus, a proxy hides the client from the servers, and a reverse proxy hides the server from clients. We will see many examples of proxies and reverse proxies, and discuss why we would want them.

5.2 IP and MAC addresses

An *IP* (short for *internet protocol*) address is the address of a device connected to the internet. We will discuss *IPv4* and *IPv6*. An IPv4 address is of the form x.x.x.x, where each x is a number between 0 and 255 inclusive. Thus there are $255^4 = 2^{32}$ possible IPv4 addresses. It turns out that there are many more devices connected to the internet (recall that these are called *hosts*) than possible IPv4 addresses. The solution for the future is IPv6, which uses 128-bit addresses (and thus has 2^{128}

possible addresses in total). Protocols that the internet runs generally support both IPv4 and IPv6. Note: each device does have an address, called the *MAC* (media access control) address, that consists of 48 bits and is supposed to be unique.

The current solution to the scarcity of IPv4 addresses is that not everyone gets a fixed IPv4 address. We have *public addresses* and *private addresses*. A public address is globally unique, whereas the private addresses (which are always in the ranges 10.0.0.0–10.255.255.255, 172.16.0.0–172.31.255.255, 192.168.0.0–192.168.255.255) are re-used. (The address 127.0.0.1 is special, called *localhost*, and is used to directly serve/connect to the host itself without going through the LAN.) Each internet service provider (ISP) has a set of public IPv4 addresses that it owns. End users first connect to their local area network (LAN), where devices are identified by private IP addresses. In the LAN, there are some devices that are connected to the *wide-area network* (WAN), i.e. the "true internet". These devices (which may or may not also be the WLAN router) have both a public and a private IP address, and perform *network address translation* (NAT). Essentially, to communicate with the outer world, you send your packets to the NAT box using the LAN, and the NAT box forwards them to their address, returning the packets from the outer world (that were addressed to you) back to you. Formally, a NAT box translates a private IP address + a *port number* (you can think of an IP address as the address of a building, and the port number as the apartment number) to a public IP address (namely, its own public IP address) + a port number. Because of NAT, public IP addresses are not routable, meaning that you cannot send a message to someone's public IP address. Hence establishing communication without either party having a public IP address is actually not completely trivial.

You can connect to your router using a LAN address which usually can be found on the router. This will open an interface where you can configure the router. Typical LAN addresses for configuring the router are 192.168.0.1, 192.168.1.1, 192.168.100.1 etc.

Note: when running a virtual machine, you can either connect to the network using the host machine as a NAT box. or you can directly connect the VM to the network, which is called *bridging*.

5.3 Network and transport layers

Communication over the internet happens at different levels of abstraction: over the cable at the physical level and between programs running on different computers at the highest (application) level. The *OSI network model* describes 7 levels of abstraction: each level uses the levels below it as a black box. IP addresses are a feature of Level 3, called the *network layer*.

In the network layer, the hosts exchange messages called *packets*. Each packet has, among other things, the source and destination IP addresses, checksums, payload (actual information being transmitted), a description of the payload etc. When receiving a packet, the hosts forward it towards the address of the recipient (unless it is addressed to them): this is called *routing*, and is done using the internet protocol (IP). On average, a packet gets passed around ≈15 different hosts before reaching its destination.

The transmission control protocol (TCP) sits in Level 4 of the OSI model (called the transport layer), and is responsible for reliable end-to-end communication between applications running on different hosts (identified by an IP address + port number). For example, IP packets can arrive out of order, irregularly, duplicated, or not at all: TCP then detects the missing packets and arranges the received packets back into their correct ordering

An alternative to TCP is user datagram protocol (UDP). It trades off some reliability for faster performance. UDP is used in contexts where dropping a small number of packets is preferable to waiting/delays, e.g. video streaming and online gaming.

5.4 Programming examples and homework

You can view the IP addresses of your Linux machine (typically one IPv4 and one IPv6) using `hostname -I`. For full information (usually too much) about all network interfaces, use `ip address`.

You can use `nmap` (short for "network mapper") to check which ports are open which servers are

running at a given address. For example, at the time of writing, running `nmap mpi-sws.org` produces

```
PORT STATE SERVICE
23/tcp filtered telnet
80/tcp open http
139/tcp filtered netbios-ssn
443/tcp open https
445/tcp filtered microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 6.71 seconds
```

You can also use `nmap` to discover all hosts connected to your network: `nmap 192.168.0.0/16`, for example, scans all IPv4 addresses that begin with 192.168.0, which is what my router assigns to connected hosts.

Homework 1) Carefully study the OSI model. 2) We mentioned that, due to NAT, direct peer-to-peer communication between to hosts neither of which has a public (fixed) IP address is not possible. Then how do WhatsApp/Telegram etc. work? 3) Connect to your router at home. Check which information is available and what you can configure. 4) Study the full capabilities of `nmap`.

6 Network protocols for security and privacy

6.1 Transport layer security (TLS)

TLS (see Request for Comments (RFC) 8446 for the exact specification of TLS 1.3, which is the current version) sits between the transport and application layers of the OSI model. It provides (server) authentication, encryption, and key exchange to the application layer. TLS is a replacement for SSL (secure sockets layer), which turned out to be not sufficiently secure, but serves the same purpose. (Nowadays, when people/the operating system says "SSL", what they mean is TLS.) TLS operates over TCP; in particular, it first opens a TCP connection from the client to the server which involves the usual SYN, SYN-ACK, and ACK messages. The TLS protocol has two steps. (What is written below is not 100% exact; read the TLS specification for that.)

Step 1: handshake. The client sends (i) the list of TLS versions and (ii) the list of encryption, hashing, and key exchange methods (collectively called the *cipher suites*) that it supports. The server replies with (i) the TLS version and the cipher suite that it has selected and (ii) a certificate from a certificate authority (CA) that verifies its identity. (All communication by the server is signed using their public key, which the client decides to trust or not after checking the certificate.) In the next step, the client and the server perform *elliptic curve Diffie-Hellman* (ECDH) key exchange to establish the *session keys* (e.g., for symmetric encryption and computing MACs). These keys are derived from publicly exchanged information. Finally, the client and the server communicate to each other that the handshake was completed successfully.

Step 2: communication. The communication happens over TCP. It is encrypted using a symmetric AEAD (Authenticated Encryption with Associated Data) method, e.g. AES-GCM, and the corresponding session key. This provides confidentiality, authentication (of server's identity–in vanilla TLS, the server does not know to whom it is talking to), integrity (this is both for server's and client's messages, thanks to AEAD), and forward secrecy.

How does the client verify the certificate? The client trusts a *root CA*, whose information is stored locally in the computer. It builds a *chain of trust* from the root CA to the CA that signed the certificate. In Fig. 1, for example, we see that the certificate for the website is signed by the intermediate CA "GEANT TLS RSA 1", whose own certificate is signed by the root CA "HARICA TLS RSA Root CA 2021". Thereafter, the client checks the expiry date of the certificate as well as the known *revocation lists*, which store the certificates that have become invalid.

6.2 Hypertext transfer protocol (HTTP)

We now discuss the hypertext transfer protocol, one of the foundational pillars of the world wide web. A *hypertext* is an HTML file with links to other files. Nowadays, HTTP is used to transfer all sorts

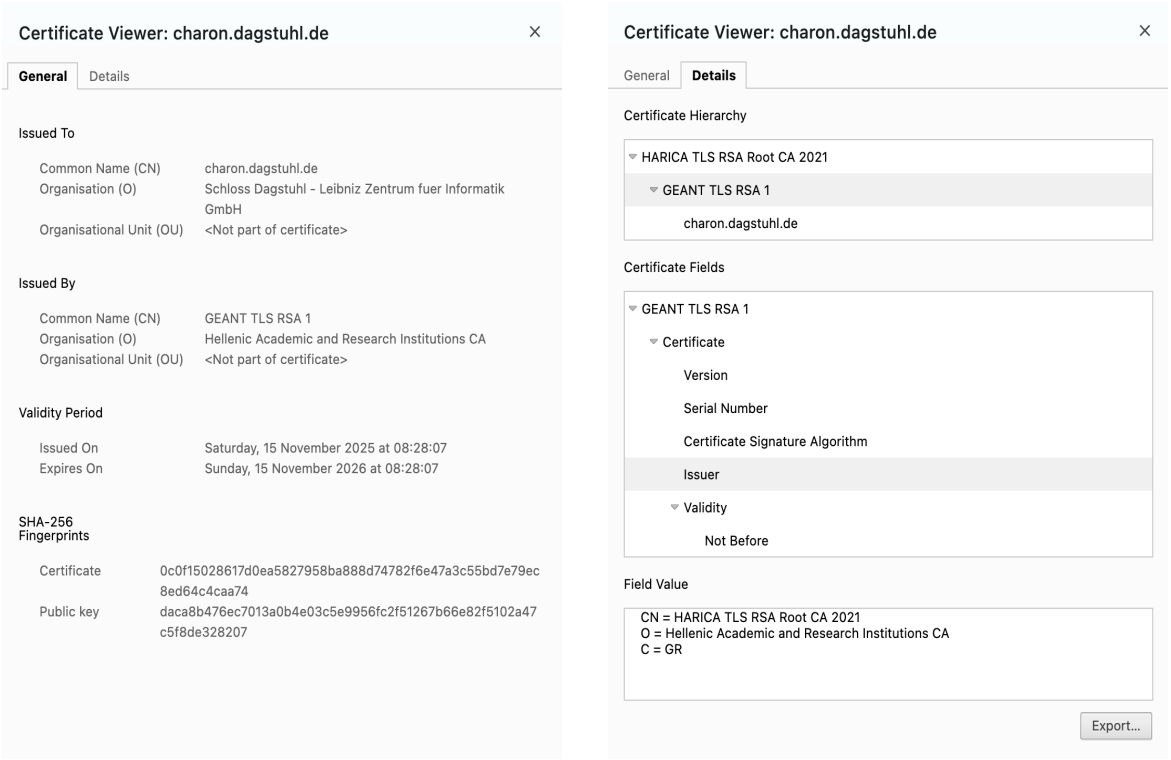

Figure 1: TLS certificate of a website. CN ("common name") is the name of the website, O ("organisation") is the name of the organisation that owns the website and the certificate, and C is the country code for the said organisation.

of information (not only hypertexts) from servers to clients and vice versa.

In HTTP, there is no encryption or authentication: it is fundamentally very unsafe. Despite this, it was widely used (from its invention in 1990) until 2010. (Web servers do still respond to HTTP requests, but they usually redirect the client to use HTTPS instead.) Thereafter, as the public started to become more aware of security issues, HTTPS (=HTTP Secure) gradually became the norm. At the moment, HTTPS is HTTP over TLS, i.e. it performs a TLS handshake between the client and the server before transmitting data using HTTP, but in the past it used SSL. The protocols HTTP/HTTPS are managed by the Internet Engineering Task Force (IETF). At the time of writing, HTTP is specified in Request For Comments (RFC) 9110.

A *uniform resource locator* (URL; a URL is less general than a *uniform resource identifier* (URI)) is a string of the following form

scheme ":" ["//" authority] path ["?" query] ["#" fragment].

It identifies a resource (e.g., a file) by specifying how to reach it. The parts in square brackets are optional. The space character needs to be written as %20. For example, in the URL

https://example.com/api/info?username=max%20planck

the scheme is HTTPS, the authority is example.com, the path is /api/info, and the query has a single parameter called username with the value "max planck". Such a URL can be used to get all the information that the server has about the user called "max planck".

Let us further discuss the components of a URL.

- The scheme can be, among others, https, http, ftp, and mailto. We will only focus on https. In this case, the authority part is mandatory. If you omit the scheme, the browser will fill it in, preferring HTTPS over HTTP whenever possible.
- The authority part is itself of the form

  [userinfo "@"] host [":" port]

where host is a registered name or an IP address, port is the port number, and the user information is typically of the form username:password. The latter is rarely used these days: more commonly, the client puts their *API key* into the URL, and the server recognises the entity making the request. If host is an IPv4 address, then we just give it, e.g. the host 8.8.8.8:9000 has the port 9000 at the address 8.8.8.8 as the authority. With IPv6, however, because the address already contains the : symbol, we have to enclose the address in square brackets as in

https://[2001:db8:85a3:8d3:1319:8a2e:370:7348]:8080.

- The path looks like an absolute Linux path (it always starts with /), but it may not correspond to a physical location: in the example above, the server does not need to contain a file called "info" inside of a folder called "api". Instead, the server will look at the path and just interpret from it what the client wants. (We mention that the path *can* be a physical path in the file hierarchy of the server.)
- The optional query part contains the values of the arguments that the server expects. The syntax of this part is

  ?argument1=value1&argument2=value2 $\cdots$.

- Finally, the fragment part selects an HTML elements from the response of the server (identified by, e.g., the id of the HTML element). For example, at the time of writing,

  https://en.wikipedia.org/wiki/URL#Internationalized_URL

  identifies the element

  ⟨h2 id="Internationalized_URL"⟩Internationalized URL⟨/h2⟩

  inside the HTML file for https://en.wikipedia.org/wiki/URL.

In HTTP/HTTPS, the clients send GET and POST requests (there are also other types of requests too), and the server sends back a response. When you type a URL into your browser, after establishing encrypted communication with the server (resolving the domain name into an IP address first if necessary), the browser creates a GET request, which has the following shape:

GET path[?query] http-version
Host: host
User-Agent: information-about-the-client-program
Accept: what-kind-of-a-thing-we-want
empty-line
optional-body

The first line is called the request line. The following lines until the empty line are the headers, the part before : being the name of the header. A GET request can but need not have a body. There are other possible headers: these can be used, for example, to specify what to do with the connection after the request is served, or to authenticate the client either using the username:password part of the authority or sending a *session cookie* within the request. As an example, connecting to my website (toghrul-karimov.github.io) from my machine produces the following GET request:

GET / HTTP/1.1
Host: toghrul-karimov.github.io
User-Agent: curl/8.7.1
Accept: */*

Notice the empty line above. The fragment part of the URL is handled at the end, after the server has responded. The response of the server contains a status code (for example, 200 for "OK" and 404

for "Not Found"; the server is free to respond as it pleases), some headers (e.g., information about the server), and finally the actual response to the query (e.g., the contents of an HTML file).

Now let us look at POST requests. These look almost exactly the same as GET requests, except they have "POST" instead of "GET" in the request line, and, unlike GET requests, commonly contain a body. We can still use query parameters to pass information, but the body has the advantage that its size and structure is completely unrestricted. POST requests typically cannot be sent using a browser: instead, we will use the command-line tool curl (cURL, inspired by "see-U-R-L", but pronounced curl). For example,

curl –request POST –data "Was born in 1998" "toghrul-karimov.github.io?name=toghrul"

sends the following POST request:

POST /?name=toghrul HTTP/1.1
Host: toghrul-karimov.github.io
User-Agent: curl/8.7.1
Accept: */*
Content-Length: 16
Content-Type: application/x-www-form-urlencoded

Was born in 1998

The command curl has a range of interesting options. For example,

curl –write-out "%time_total" ···

prints the time it took (including the TLS handshake if using HTTPS) to complete the request, and

curl –max-time m ···

sets an m-second time limit for completion of the request. When the client drops the connection, the server typically still completes the request: this can be important in DoS/DDoS attacks. Finally,

curl –verbose ···

prints detailed information about the exchanges between the client and the server, including the GET/POST requests created.

6.3 Secure shell (SSH)

SSH is a protocol for remotely logging into your account. It is described in `https://datatracker.ietf.org/doc/html/rfc4253`. You can run both an SSH server and an SSH client on your Linux/MacOS/Windows machine. To remotely log in to the machine with IP address `address` (recall our discussion about public and private IP addresses) as the user `user`, assuming the said machine is running an SSH server, you do `ssh user@address`. By default, the SSH server runs on port 22. The server then sends its public key, and starts key exchange with the client: all messages by the server are signed with its key. After the key exchange, the client decides whether to trust the server or not, which is subtly different from TLS. If the client decides to trust the server, they switch to (symmetric-key) encrypted communication. At the next step, the server asks the client for the password of `user`. If authentication is successful, then the client gets access to a terminal run by `user`.

6.4 Programming examples and homework

In a Linux system, you can view the list of all root certificate authorities and their public keys etc. (warning: this has nothing to do with PGP: this is about the centralised model used by TLS) under `/etc/ssl/certs`. The certificates in the X.509 format (which defines what information is present in the certificate) and are stored as `.pem` files. You can extract human-readable information from such a file using `openssl x509 -in file.pem -text`.

Homework 1) Learn about the POODLE attack on SSL (and earlier versions of TLS). 2) Investigate how to configure an SSH server on your machine. 3) Log in to your host machine via SSH from your guest OS, and vice versa. Use bridging to directly connect the VM to the internet and temporarily turn off firewalls if necessary.

7 Higher-level tools for security and privacy

7.1 Firewalls

A *firewall* is a program that filters out internet traffic (e.g., allows only certain packets to pass) according to a set of rules. It typically sits between the public and the private networks (for example, a NAT box can also act as a firewall), or in the host machine (for example, the operating system can forbid certain users from establishing certain connections). Firewalls can also operate in the *application layer* of the OSI model, in which case they understand the protocols/applications that are being used at a very high semantic level. These are called *proxy* firewalls, as opposed to the simpler *packet-filtering* firewalls that operate in the network layer. A packet-filtering firewall can be implemented in Linux using `nftables`.

7.2 Virtual private networks

We now briefly discuss how network layer virtual private networks (VPNs) work. (There are other kinds of VPNs too.) Suppose I want to send a packet with payload m to the IP address a. Then the intermediate hosts in my LAN (and possibly other hosts in the WAN too) will see the address of the recipient of my packet as well as its contents. (Assuming we are dealing with IP rather than IPsec, there is no encryption at the network layer–it is typically performed at the higher levels.) To hide all information about my packets, I first connect to a VPN server outside of my LAN. Then I encrypt $m \,||\, a$, and send the resulting ciphertext to the VPN server: this is called *encapsulation*. Now the intermediate hosts do not know anything about m and a. (But they can, through various means, detect the fact that I have connected to a VPN server.) The VPN server decrypts my message, and sends m to a from its own name. This way, I *tunnel* out of my LAN. The VPN server sends the packets from a back to me by reversing the process described above.

7.3 Tor

7.4 End-to-end encryption

Emails are not end-to-end encrypted: the service you use does have access to the plaintexts, and routinely serves them to governments upon legally requests. Another issue is that it is easy to at least attempt to *spoof* emails: email operates using the simple mail transfer protocol (SMTP), and one can easily send an email with the "from" field displaying any chosen address, though, assuming the receiving mail server has good security, the forged email will not be accepted. So what to do?

Ideally, all important emails should be digitally signed, which confirms the identity of the sender and the integrity of the message. (In serious contexts, you should treat all unsigned email as malicious.) There are two main ways to do this (Fig. 2). The first one is the secure/multipurpose internet mail extensions (S/MIME) protocol. Here, you create a pair of public and private keys, and apply to a certificate authority for certificate that shows "this public key belongs to the owner of this email address". After stronger validation, these certificates can also certify the *personal identity* of the owner of the email address. Governments and companies, for example, often mandate the use of S/MIME certificates. Once you have a certificate, you add it to your email software. Then the messages you send are digitally signed by your certified key pair.

The second approach is the PGP (Pretty Good Privacy) protocol. This one is free and open source: usually, either you or your company has to pay for an S/MIME certificate. You generate a key pair yourself, and add it to the OpenPGP key database (called a *key ring*). Then you communicate this key to the world, e.g. on your personal website. Others, when receiving an email from you for the first time, verify that this key really does belong to you. This step is either performed "in the real world", or with the help of the PGP software: it records the known chains of trust, and you can see that someone you trust has already trusted this key. Once you decide to trust the key, it is added to the chain of trust, and you don't have to do anything the next time.

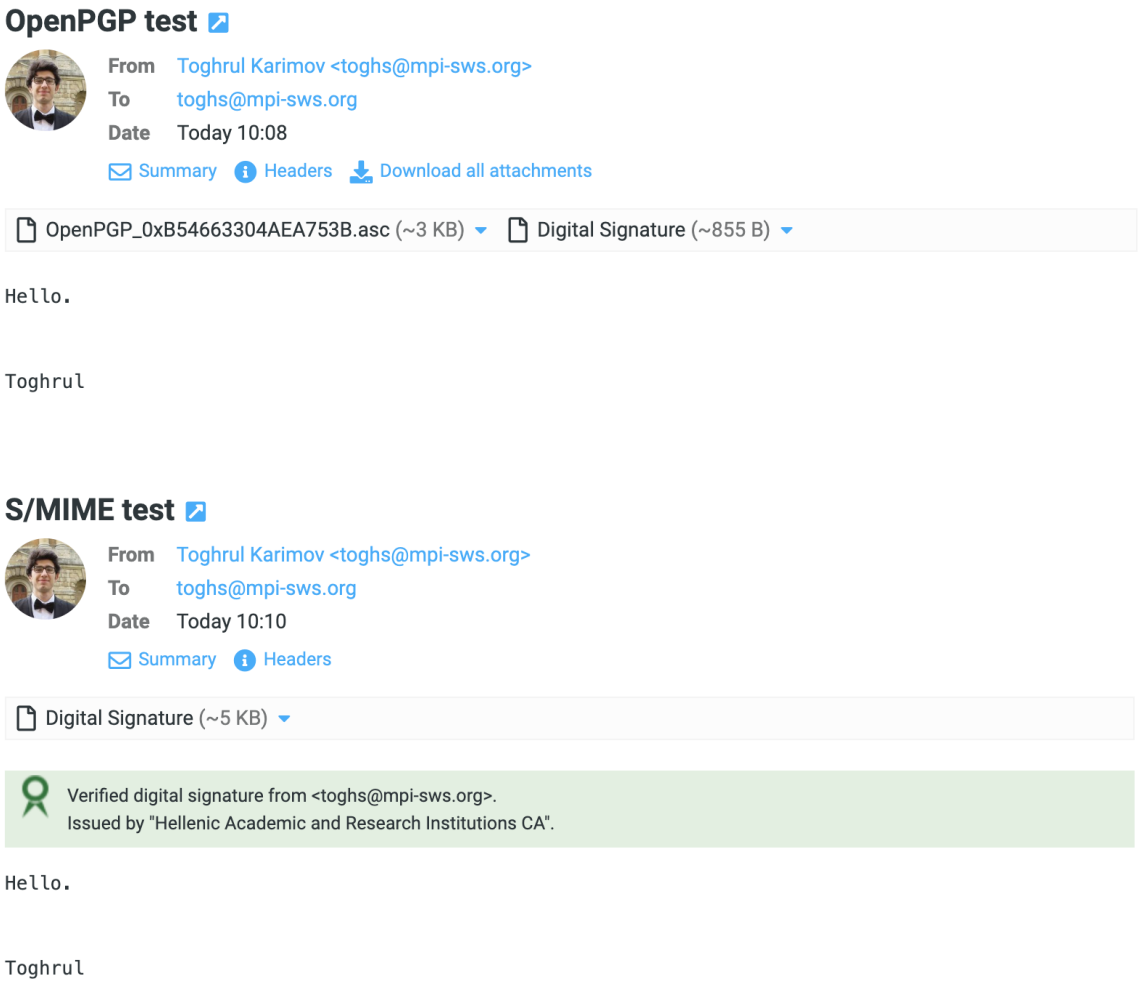

Figure 2: S/MIME and OpenPGP-signed emails

If you know the public key of the recipient of your email, you can send end-to-end encrypted emails; this is the only true way of secure and private communication over the internet. Email software like Thunderbird make encrypting/decrypting/signing emails very easy. Both of these approaches also keep track of which keys have been revoked.

7.5 Programming examples and homework

A well-known toolbox for encryption, decryption, and digital signatures is PGP, which stands for "pretty good privacy". In Linux systems, it is implemented by the open-source program `gpg`. It is often used to sign published software, as well as to sign/encrypt emails. For example, suppose we download `hashcat` sources from the official website: there we have, in addition to a `.tar.gz` file, we have a `.asc` file. The latter is the hash of the true file, signed with the private key of the publisher. Its purpose is to certify that the file has not been tampered with. In addition, the download page provides a *key id* and a *digest* for the public key of the publisher: we need this key to verify the signature. (At the time of writing, on the official website we have "Signing key on PGP keyservers: RSA, 2048-bit. Key ID: 2048R/8A16544F. Fingerprint: A708 3322 9D04 0B41 99CC 0052 3C17 DA8B 8A16 544F".) The fingerprint is (yet another) hash of the true public key, and is useful because keys are typically long and cannot be easily compared by eye. So, to verify our download of the hashcat files, we first run `gpg --recv-keys 8A16544F`, which fetches the full (public) key from the key server. The keys fetched are then stored in the home folder of the user. We can then check the fingerprint of the key using `gpg --fingerprint 8A16544F`, which prints A708 3322 9D04 0B41 99CC 0052 3C17 DA8B 8A16 544F. Note that this matches the fingerprint supplied by the publisher. Finally, we can verify the integrity of the file by running `gpg --verify path-to-the-.asc-file path-to-the-downloaded-file`. I

get "Good signature from ..." and at the same time "WARNING: This key is not certified with a trusted signature!" That is, the signature does indeed belong to the entity owning the public key, but `gpg` does not vouch for the true identity of the key owner. I can, in my local settings, decide to trust this key. Finally, I can view more information about the contents of a signature (e.g., if present, the public key for verification, the hashing algorithm used etc.) by running `gpg --list-packets path-to-file`.

Homework 1) Install Mozilla Thunderbird, and configure your digital signature using OpenPGP. 2) Send and receive end-to-end encrypted emails. 3) Look up the `torrc` configuration file, and how you can modify it to force an exit node in the country of your choice.

8 Web applications and their security

A web application (e.g., YouTube, WhatsApp Web, or any website for that matter) has three logical parts: the client part (=the front end), the server part (=the back end), and the database. (The database can also be viewed as part of the back end.) The client part is the code (can be written using HTML, CSS, and JavaScript) that runs on client's machine, e.g. in the browser of your laptop. It is responsible for displaying the information received from the server, as well as communicating the user input to the server. The server part consists of the code (can be written in Python, C#, PHP, Go, and many other languages) that serves the application to the client. This part is responsible for the intensive computations, including working with the database. The database is managed by a database management system (e.g., PostgreSQL, MySQL, Neo4j), and the server usually interacts with it using a variant of the simple query language (SQL), although SQL-free options also exist.

It is advisable to have a reverse proxy (e.g., nginx) in front of the server part of the application. This hides the application code from the outside world, takes care of the certificates for TLS, and can analyse incoming HTTP requests (e.g., blocking or throttling an IP address if it sends too many/malicious requests). You can get a TLS certificate for free for your self-hosted application using Let's Encrypt.

An average cyberattack is not centred around cracking fundamental cryptographic primitives like encryption or hashing: rather, it exploits bad programming and lax security practices. We next discuss a few of the most well-known attacks on web applications. (See "OWASP Top 10:2025" `https://owasp.org/Top10/2025/` for the latest top 10 cybersecurity issues for web applications.)

8.1 SQL injection (SQLi) attack

Suppose our application has a database with a table called `users`, which has columns `name` and `password_hash`. For simplicity we assume that the application uses the same salt string for all users. Suppose that the front end sends the back end two strings called `username` and `password` that store the credentials entered by the user. Let's say the back end checks whether to grant access or not as follows. (We are using `sqlite` with Python.)

```
query = f"
  SELECT * FROM users
  WHERE name = '{username}' AND password_hash = '{hash(password, salt)}'
"
<execute the query and check if the answer is empty or not>
```

Then the attacker can submit `username = " ' OR 1 = 1 -- "` and any password. The query constructed will be

```
query = "SELECT * FROM users WHERE name = ' ' OR 1 = 1 -- ' AND ···"
```

but the part after `--` will be interpreted as a comment and discarded. So in the end,

```
query = "SELECT * FROM users WHERE name = ' ' OR 1 = 1"
```

and returns the whole table, which is non-empty. Thus the attacker is granted access, without knowing a username or password. To prevent the SQL injection attack, raw strings should never be used to

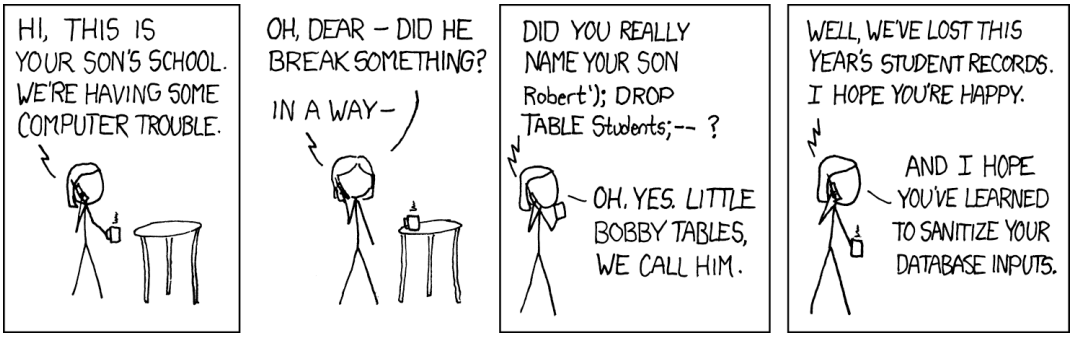

Figure 3: Xkcd about SQL injection.

construct queries. Instead, the contents of `username` and `password` should be interpreted as values:

```
execute(
  "SELECT * FROM users WHERE name = ?  AND password_hash = ?",
  (username, hash(password, salt))
)
```

This executes the query, substituting the correct values in place of ?.

8.2 Cross-site scripting (XSS) attack

In this case, the attacker first injects JavaScript code into the website. For example, the website might have a forum where the users can post comments. Suppose the attacker posts `<script>some code</script>`, and the website stores it in its database. When another user requests the webpage that contains the post by the attacker, the server sends an HTML file that contains `<script>some code</script>` inside it. When the browser renders the HTML file, it can interpret this as the instruction to run `some code`. If successful, the attacker can run any JavaScript code inside the browser of the user that would normally be permitted by the browser. When getting XSS attacked, you might see the warning "this site is trying to load malicious scripts", though this might occur due to other reasons too. To prevent XSS attacks, there is a list of things one should be careful about (see `https://owasp.org/www-community/attacks/xss/`), but the main idea is that you should not let text be run as code.

8.3 Denial-of-service (DoS) attack

In a DoS attack, the attacker overwhelms the resources of the system so that they become unavailable to legitimate users. For example, a website that can handle 100 requests per second becomes unavailable when it gets flooded with thousands of requests per second. To prevent a DoS attack, the server can blacklist the IP address that is sending too many requests and ignore the incoming packets coming from that address: this can be done, for example, at the network level, similarly to how a packet-filtering firewall works. This is where *distributed* DoS (DDoS) attacks come into play: in this case the attacker sends a large number of requests from many different IP addresses. The standard way to protect the website from such attacks is to use a protection service like Cloudflare. Then what happens is that the real IP address of your servers is not exposed to the internet: the incoming traffic first goes through the servers of Cloudflare, which use sophisticated techniques (involving layers 4 and above of the OSI model) to detect and block DDoS attacks. This is yet another example of a *reverse proxy*: the client thinks it is talking to the server, but in reality it is talking to someone else who passes the communication on to the real server. For comparison, a *proxy* sits between the client and the server, and communicates with the server on behalf of the client. A proxy hides the identity of the client from the server, whereas a reverse proxy hides the real server from the clients.

8.4 Programming examples and homework

You can find a demonstration of a basic SQL injection attack at `http://toghrul-karimov.github.io/assets/lecture-notes/sqli.py`. You can run it on your local machine at your desired port

using Python and Flask as

python3 sqli.py --host 127.0.0.1 --port 8001

The database of registered users has just one table called `users`, with just one column called `name`. It contains the entries "Alice", "Bob", "Charlie". As written, security measures of the code are so bad that you can input special strings that will cause a Python error and display the underlying code to you in the process.

For a demonstration of an XSS attacks and an API, see `http://toghrul-karimov.github.io/assets/lecture-notes/xss.py`. You can run it on localhost exactly as above.

A simple DoS attack on a simple Flask server (i.e., not the production-grade `gunicorn` server) serving the XSS demonstration website above can be implemented as follows. The Flask server can handle one request at a time, on a first-come, first-serve basis, and it will finish serving the request even if the client drops the request. We can first perform many POST operations, to fill up the database. This makes the GET requests slower; let's say we bring it up to 0.5s. We can then simply make, inside a loop, GET requests using `curl` that time out after 0.1s. This will explode the stack of requests the server has to address, and make the website unavailable even after the attacker stops sending GET requests.

Homework 1) Look up what OWASP is. 2) Study how you can serve a website over LAN. 3) Look up `nginx` is and what it does (reverse proxy, DoS protection, handling TLS certificates). 4) Compare web scraping with a DoS attack.

9 Common types of cyber-attacks

Below we discuss a few more types of frequently occurring cyberattacks. We mention that the term *zero-day* refers to a previously unknown software vulnerability that attackers exploit: in this case the defenders have zero days to prepare for the attack.

9.1 Phishing and social engineering

This is a cyberattack where an attacker pretends to be a trusted person to trick someone into revealing sensitive information, clicking a malicious link, or performing some other unsafe action. My email is available online, and bots scrape it to send me phishing emails all the time: they pretend to be someone from my workplace (sometimes even my boss) that needs something from me. Sometimes the email originates from my university: someone's email is compromised and then used to send phishing emails to all contacts from a usually trusted address. Phishing is considered one of the most effective cyberattacks–its cost to the attacker (especially with the advent of LLMs) is practically zero, and it is much easier to trick humans than crack established security primitives. Types of phishing include but are not limited to email phishing, voice phishing (also called vishing), spear fishing (targeted fishing at high-ranking employees: spear fishing is used to catch big fish in fishing), and evil twin phishing (e.g. setting up a fake wi-fi hotspot). To prevent phishing attacks, the users should verify and treat all communications with scepticism. The organisation should mandate *multi-factor authentication* (MFA), so that there is a second layer of protection in case a password gets stolen (which more often than not happens through a phishing attack). At my university, if an email address sends too many messages in a short period of time, it is suspected of being compromised and gets blocked.

Fig. 4 shows a phishing email that I recently received. (I first inspected the link, then opened it inside a disposable, clean virtual machine in order to minimise any possible damage: this is known as *sandboxing*.) Note that there is no digital signature or a certificate attached to the email. However, *someone's* signature could have also been attached, in which case it would be our job not to trust the signature if it is unknown. Even if there were a signature attached from someone we trust, we should separately contact that person and verify, and refuse to type in password if necessary.

9.2 Malware (=malicious software)

A *virus* is a malware that attaches itself to legitimate files, and is spread when the file is executed. A *worm* is malware that replicates itself and spreads across networks without the users' knowledge. Example: WannaCry (see below). *Trojan* refers to malware disguised as legitimate software, that

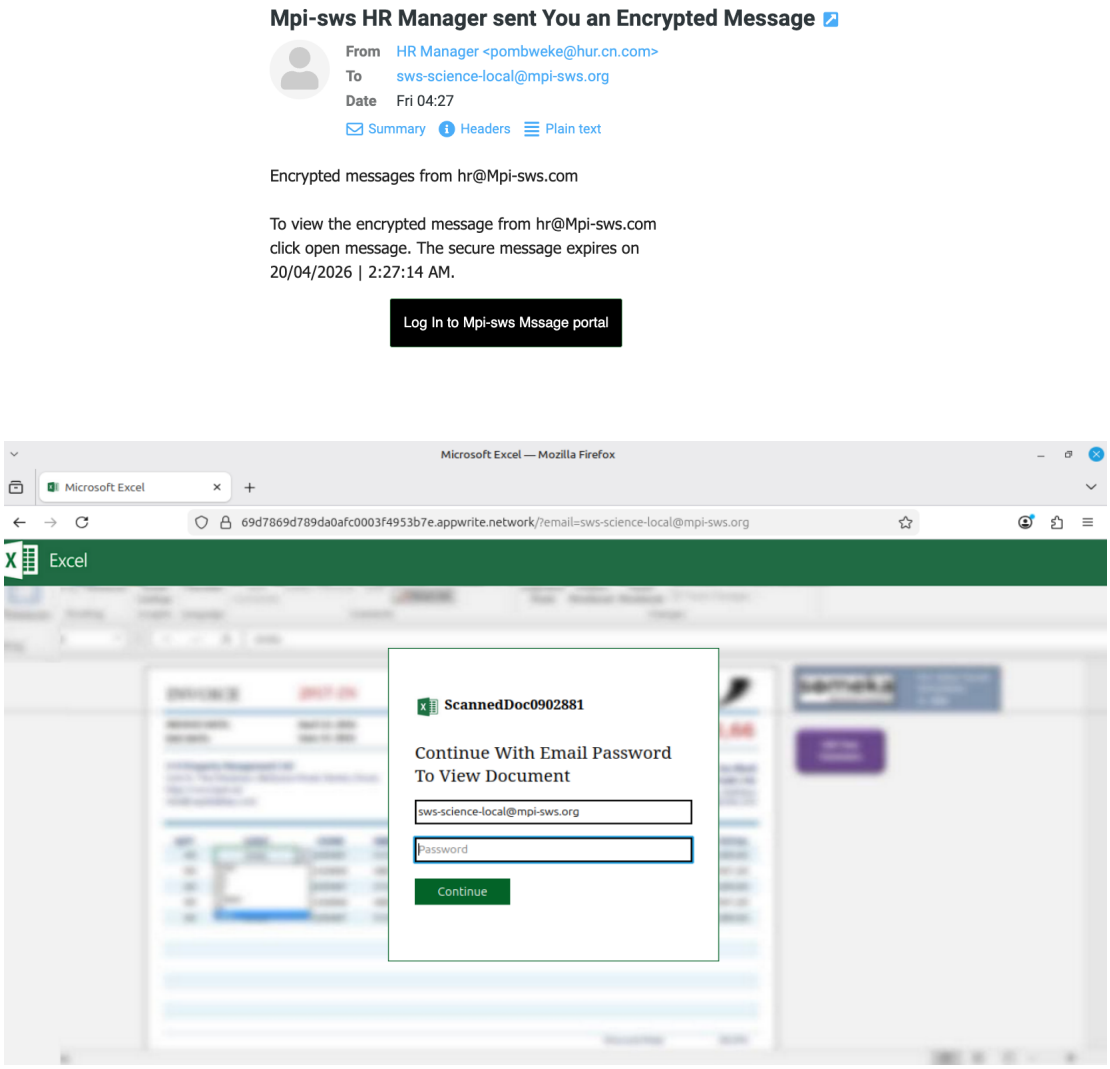

Figure 4: A phishing email and the website it links, where you are asked for a password.

tricks users into installing it. *Ransomware* is malware that encrypts files and demands ransom to restore access. *Spyware* is malware that secretly observes and records informations about the user.

9.3 Supply chain attacks

9.4 Few case studies

1. `https://www.ic3.gov/PSA/2026/PSA260407`. Attack on DHCP settings of affected TP-Link routers that modifies the address of the DNS server, so that DNS requests go to a server controlled by the attacker. The DNS server gives fake addresses for well-known sites. If the user proceeds despite certificate warnings, they input their data into a fraudulent website (man-in-the-middle attack).

2. `https://techcrunch.com/2026/01/05/hacktivist-deletes-white-supremacist-websites-live-on-stage-during-hacker-conference/` `https://www.youtube.com/watch?v=lJsS8lqCpwU&t=598s`. Hacktivist Martha Root steals and wipes all data from morally questionable websites on live TV, using URL manipulation.

3. `https://github.com/axios/axios/issues/10636` A monumental supply-chain attack on a very popular `npm` library that uses, among other tricks, a prompt injection.

4. `https://red.anthropic.com/2026/mythos-preview/` Anthropic has released a new AI model that is supposedly extremely good at finding previously undiscovered security vulnerabilities in

well-established tools and protocols.

5. https://www.cloudflare.com/en-gb/learning/security/ransomware/wannacry-ransomware/

6. https://en.wikipedia.org/wiki/Stuxnet